

Что такое Git.



Что такое Git

Контроль версий, также известный как управление исходным кодом, — это практика отслеживания изменений программного кода и управления ими. Такая система помогает командам разработчиков работать быстрее и эффективнее.

Мы будем работать с системой Git.

Git — система контроля версий, которая позволяет отслеживать историю изменений в файлах и одновременно работать над одним проектом многим разработчикам.

Git разработал в 2005 году создатель ядра ОС Linux — Линус Торвальдс.



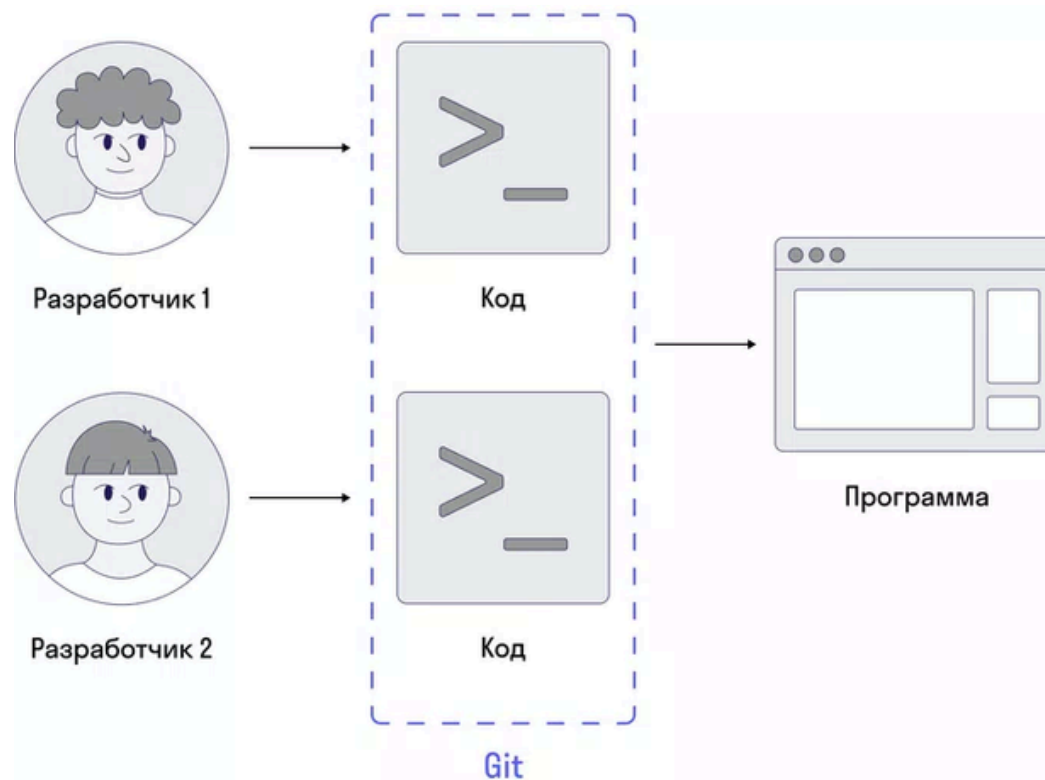
Что такое Git

Рассмотрим работу Git на примере. Разработчик написал программу, которая работает отлично. Через какое-то время приходит задача — добавить новый функционал. Разработчик пишет код — и закрывает задачу.

Но в новом функционале находят баги, поэтому просят вернуть предыдущую версию программы. Разработчик пользуется Git, поэтому ему не нужно переписывать работу.



Git хранит разные версии кода по мере его написания и внесения изменений в программу, поэтому его называют системой контроля версий.



Над проектом может работать не один разработчик, а несколько. Git позволяет видеть, когда и кто внес изменения в код, а также умеет склеивать версии кода вместе.

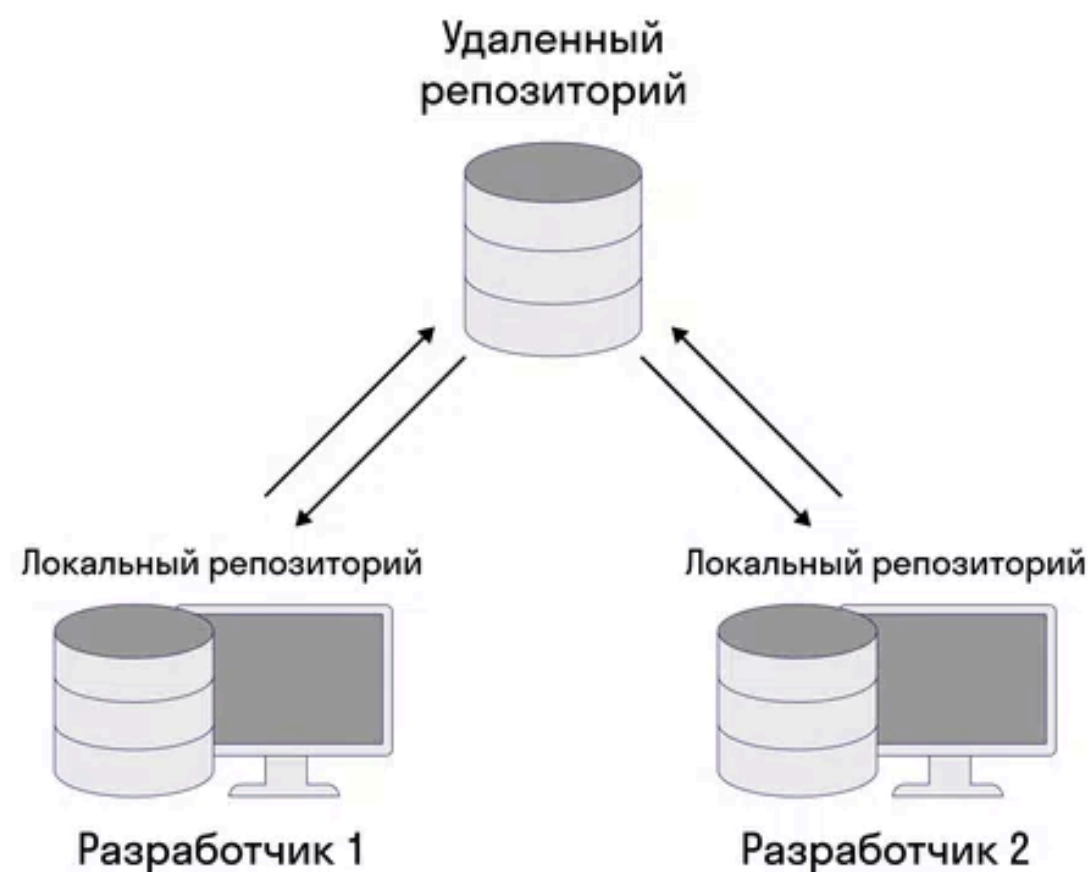
Что такое Git, Репозиторий, Коммит

Важное понятие, которое пригодится при работе с системой контроля версий, — репозиторий.

Репозиторий (от англ. repository — хранилище) — это виртуальное хранилище проекта. В нем можно хранить версии кода для доступа по мере необходимости.

Репозиторий может быть **локальным** — например, компьютер разработчика. Тогда весь код проекта хранится только на компьютере одного разработчика. Это не всегда удобно и безопасно.

Но кроме локального, есть удаленный репозиторий — код хранится на удаленном сервере или сайте.



Чтобы сохранить изменения на своем компьютере (локальном репозитории), разработчику нужно сделать commit, или «закоммитить изменения».

Коммит — это операция, которая берет все подготовленные изменения, они могут включать любое количество файлов, и отправляет их в репозиторий как единое целое.



Что такое Git, итоги

Подытожим преимущества Git:

- Git применяется для управления версиями в рамках большого количества проектов по разработке ПО, как коммерческих, так и с открытым исходным кодом.
- Система используется множеством профессиональных разработчиков программного обеспечения.
- Git работает под управлением различных операционных систем и может применяться со множеством интегрированных сред разработки (IDE).
- Git позволяет вернуться к любой версии кода из прошлого.
- В системе возможен просмотр истории изменений и восстановление любых данных.
- Git делает возможной совместную работу разработчиков без боязни потерять данные или «затереть» чужую работу.

Установка Git





Установка Git для разных ОС

Установка Git отличается в разных операционных системах. Проще всего она выполняется в macOS и Ubuntu. Они позволяют поставить Git через пакетные менеджеры. Но это не значит что Git сложно установить и для Windows:

Установка на MacOS

- Перейдите по ссылке на сайт: <https://brew.sh/>
- Введите в командной строке: **brew install git**

Установка на Ubuntu

- Воспользуйтесь привилегиями root-пользователя. Введите команду и проверьте, что пакетный менеджер содержит ссылки на последние версии Git: **sudo apt update**
- Установите Git: **sudo apt install git-all**

Установка на Windows

- Перейдите по ссылке на сайт: <https://git-scm.com/download/win>
- Установите скачанный exe-файл.

Проверка работоспособности Git

Далее я буду показывать работу с терминалом Git на базе ОС Windows:

- запустите Git Bash
- введите команду `git --version` вот что я получил при введении этой команды

```
$ git --version  
git version 2.45.1.windows.1
```

- теперь настройте Git — для своей работы ему важно знать ваше имя и почту. Эти данные подставляются в историю изменений. Только так можно узнать, кто и что сделал в проекте.

Выполняется из любой директории:

```
suz17@DESKTOP-C7FGGRN MINGW64 ~  
$ git config --global user.name "Dmitriy Sulzhits"  
  
suz17@DESKTOP-C7FGGRN MINGW64 ~  
$ git config --global user.email "dsulzhits@gmail.com"
```

Если вы указываете опцию `--global`, то настройки достаточно сделать только один раз. В этом случае Git будет использовать эти данные для всего, что вы делаете в вашей системе.

Если для каких-то проектов вы хотите указать другое имя или электронную почту, выполните эту же команду без параметра `--global` в каталоге с нужным проектом.

- Убедитесь, что всё успешно: **`git config --list`** и посмотрите на 2 последние строки

```
user.name=Dmitriy Sulzhits  
user.email=dsulzhits@gmail.com
```


Рабочий процесс



Флоу при работе с Git (Git-флоу и есть рабочий процесс в Git)

Рассмотрим порядок работы с файлами в Git на схеме.



Working directory (рабочая директория) — это та папка, где лежат файлы, которые находятся под контролем Git.

Staging area, или индекс, — область, где лежат файлы, выбранные для коммита.

Возникает вопрос: почему нельзя добавлять все измененные файлы сразу в коммит?

Такой процесс создан для удобства программистов. Дело в том, что во время разработки могут меняться и добавляться много файлов. Но это не значит, что мы хотим добавить все эти изменения в один коммит.

Со смысловой точки зрения коммит — это логически завершенное изменение внутри проекта. Его размер бывает очень маленьким, например при исправлении опечатки в одном файле, а иногда и большим, например при внедрении новой функциональности.

Главное в коммите — его атомарность, то есть он должен выполнять ровно одну задачу.

Флоу состояния файлов

При добавлении новых файлов и проверки статуса можно увидеть похожее сообщение:

```
On branch main

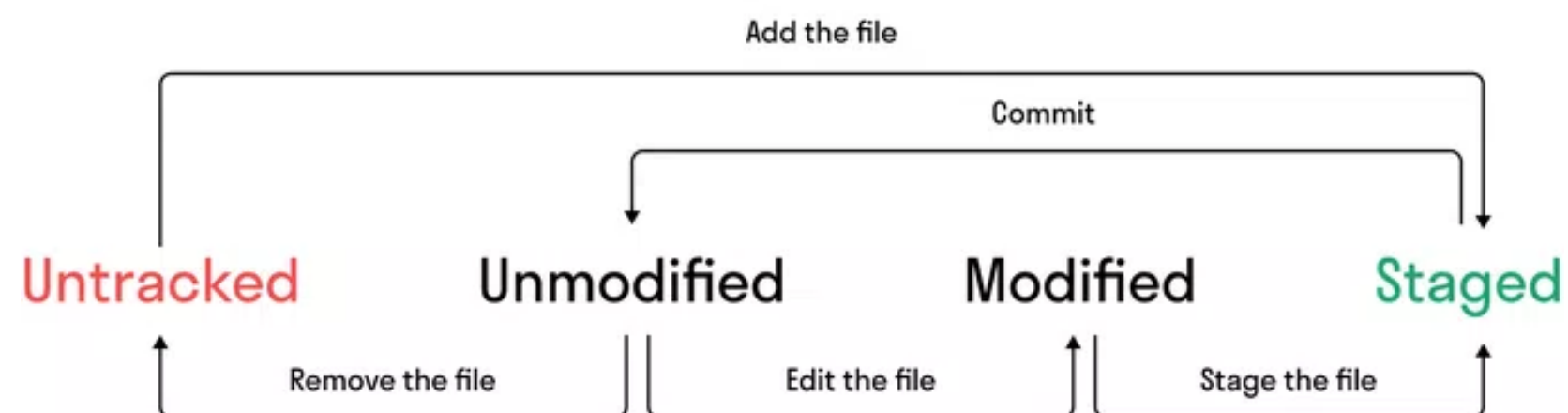
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  <название_файла>

nothing added to commit but untracked files present (use "git add" to track)
```

Git увидел, что в проекте появились новые файлы, о которых ему ничего не известно. Они помечаются как **неотслеживаемые (untracked files)**. Git не следит за изменениями в таких файлах, так как они не добавлены в репозиторий.

Рассмотрим, какие еще бывают состояния у файлов.



Staged — индекс, сюда переносятся файлы из untracked и modified. Если закоммитить файл, то он попадет в unmodified.

Modified — файл, который был изменен. Например, когда что-то дописали в коде.

Unmodified — мы уже сделали коммит, и файл сохранен в истории.

Файл **unmodified** не показывается в команде git status

, пока не будет изменен. Git показывает только файлы untracked, modified и staged.



Анализ изменений и истории

Команды для анализа изменений и истории:

- **git diff** — изменения модифицированных файлов, которые еще не были добавлены в индекс.
- **git diff --staged** — изменения файлов, которые уже добавлены в индекс.
- **git log** — список всех выполненных коммитов, отсортированных по дате добавления.
- **git show** — все изменения, сделанные в рамках одного коммита.

Анализ изменений

Во время разработки программистам часто приходится останавливаться и анализировать изменения, которые они сделали с последнего коммита.

Представьте работу над реальным проектом. Как правило, это тысячи (десятки и сотни тысяч) строк кода, сотни и тысячи файлов и иногда несколько дней работы. Если потратить несколько часов в таком проекте, очень сложно вспомнить, что и где менялось, а что еще осталось поменять.

Для просмотра изменений используется команда **git diff**. Она показывает разницу между тем, что было и что стало для файлов, которые еще не были добавлены в индекс.

Чтобы увидеть изменения с теми файлами, которые уже добавлены в индекс, выполните **git diff --staged**.

Вывод git diff содержит именно те строки, которые изменились, а не файлы целиком. Слева от них ставится знак «-» (минус), если строка была удалена, и «+» (плюс) для добавленных строк.



Анализ изменений и истории

Анализ истории

Самая простая аналитика выполняется командой **git log**. Она показывает список всех выполненных коммитов, отсортированных по дате добавления (сверху — самые последние).

Из этого вывода мы можем узнать, кто, когда и какие коммиты делал. Если коммиты оформлены хорошо, то по их описанию уже многое понятно.

У команды git log есть флаг -p, который сразу выводит diff для каждого коммита.

У каждого коммита есть идентификатор (хеш) — уникальный набор символов.

С помощью хеша и команды git show можно посмотреть все изменения, сделанные в рамках одного коммита. Пример:

```
git show cb1867b

commit cb1867b913fce5c0c6d7e2bedadc235dcc753c31 (HEAD -> main)
Author: Ivan Ivanov <ivan.ivanov@company.com>
Date:   Mon Feb 6 13:17:32 2023 +0300

    initial commit

diff --git a/file b/file
new file mode 100644
index 0000000..e69de29
```




Отмена изменений и коммитов

Основные команды:

- **git restore <file>** - отмена изменений в измененных файлах, но еще не в индексе.
- **git restore --staged <file>** - удаление из индекса и далее **git restore <file>**
- **git revert <commit_hash>** - отмена внесенных в коммит изменений и добавление нового коммита с полученным содержимым.
- **git reset HEAD~<num>** - удаление num коммитов (без num— одного) и помещение изменений в рабочую директорию.
- **git reset --hard HEAD~** - полное удаление последнего коммита со всеми изменениями.

Измененные файлы в рабочей директории

Для отмены изменений в таких файлах используется команда:

- **git restore filename** (отмена в конкретном файле имя файла должно быть указано вместе с его форматом!)
- **git restore .** (отмена во всех файлах)



Отмена изменений и коммитов

Изменения, подготовленные к коммиту

С файлами, подготовленными к коммиту, т. е. добавленными в индекс, можно поступить по-разному. Первый вариант — отменить изменения совсем, второй — отменить только индексацию, не изменяя файлы в рабочей директории. Второе полезно в том случае, если изменения нам нужны, но мы не хотим их коммитить сейчас.

Перевод изменений в рабочую директорию:

- **git restore --staged file.py**

Теперь, если нужно, можно окончательно отменить изменения в выбранных файлах:

- **git restore file.py**



Отмена изменений и коммитов

Отмена коммита через `git revert`

Вместо удаления коммита из истории проекта **git revert** отменяет внесенные в нем изменения и добавляет новый коммит с полученным содержимым. В результате история в Git не теряется, это важно для целостной истории версий и надежной совместной работы.

Отмена с помощью команды **revert** необходима, когда нужно обратить изменения, внесенные в некоем коммите, из истории проекта. Это может быть полезно, если баг появился в проекте из-за конкретного коммита. Вам не потребуется вручную переходить к этому коммиту, исправлять его и выполнять коммит нового снимка состояния — команда `git revert` сделает это автоматически.

Пример, как сделать коммит, отменяющий изменения, внесенные в коммите с хешем `cb1867b`:

```
git revert cb1867b
```

Пример, как сделать коммит, отменяющий изменения, внесенные в 3-м коммите до HEAD:

```
git revert HEAD~3
```

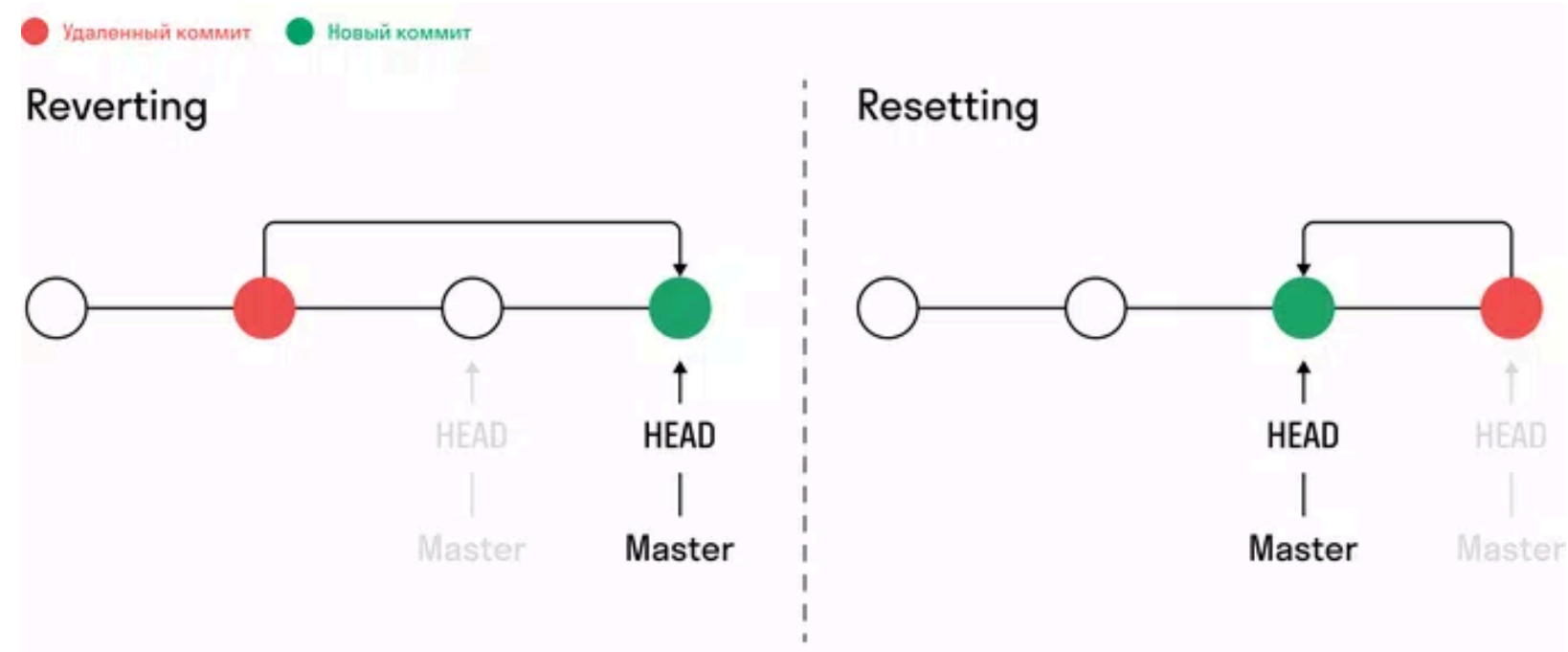

Отмена изменений и коммитов

Отмена коммита через `git reset`

Если коммит был сделан только сейчас и еще не отправлялся на внешний репозиторий или GitHub, то лучше сделать так, как будто этого коммита не существовало в принципе. Git позволяет удалять коммиты. Это опасная операция, которую нужно делать только в том случае, если речь идет про новые коммиты, которых нет ни у кого, кроме вас.

Если коммит был отправлен во внешний репозиторий, например на GitHub, то менять историю ни в коем случае нельзя — это сломает работу у тех, кто работает с вами над проектом.

Разница в работе команд `revert` и `reset` показана на схеме.



Для удаления коммита используется команда:

`git reset --hard HEAD~`

Флаг `--hard` означает полное удаление. Без него **`git reset`** отменит коммит, но не удалит его, а поместит все изменения этого коммита в рабочую директорию, так что с ними можно будет продолжить работать.

`HEAD~` означает «один коммит от последнего коммита». Если бы мы хотели удалить два последних коммита, то могли бы написать **`HEAD~2`**.

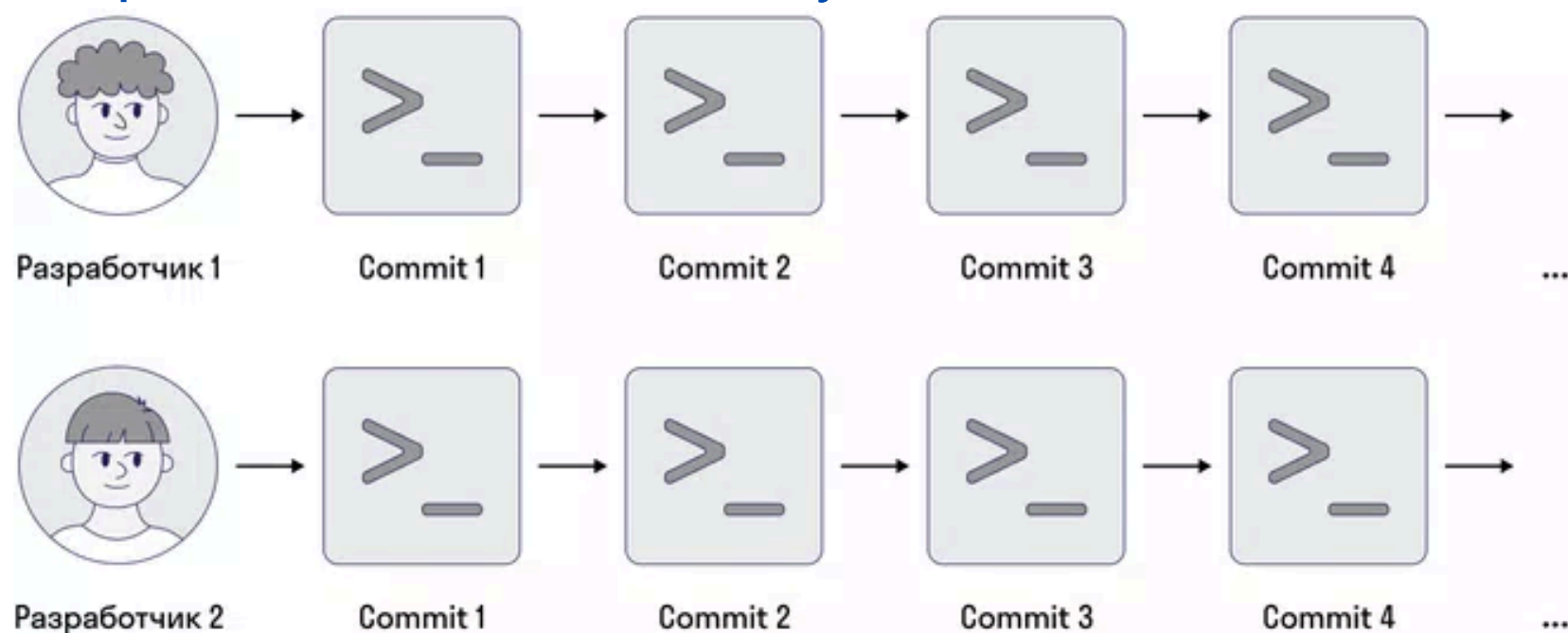
Работа с ветками



git

Что такое ветки

Когда несколько разработчиков работают над одним проектом, то чтобы не мешать друг другу, они создают ветки. Это значит, что каждый пишет свои коммиты, и эти коммиты будут сгруппированы и отправлены по соответствующим веткам.



Ветка (branch) — обособленная копия проекта, в которой хранится код и история его изменений.

В ветке можно изменять, удалять код, при этом эти изменения не затронут код в других ветках проекта.

Веток может быть сколько угодно, они создаются отдельно друг от друга.

В Git ветки — это часть процесса разработки. Если нужно добавить новую фичу или исправить ошибку, незначительную или серьезную, разработчик создает новую ветку, в которой будут размещаться эти изменения. После их разработки и тестирования новый код можно объединить с существующим.

Master - ветка

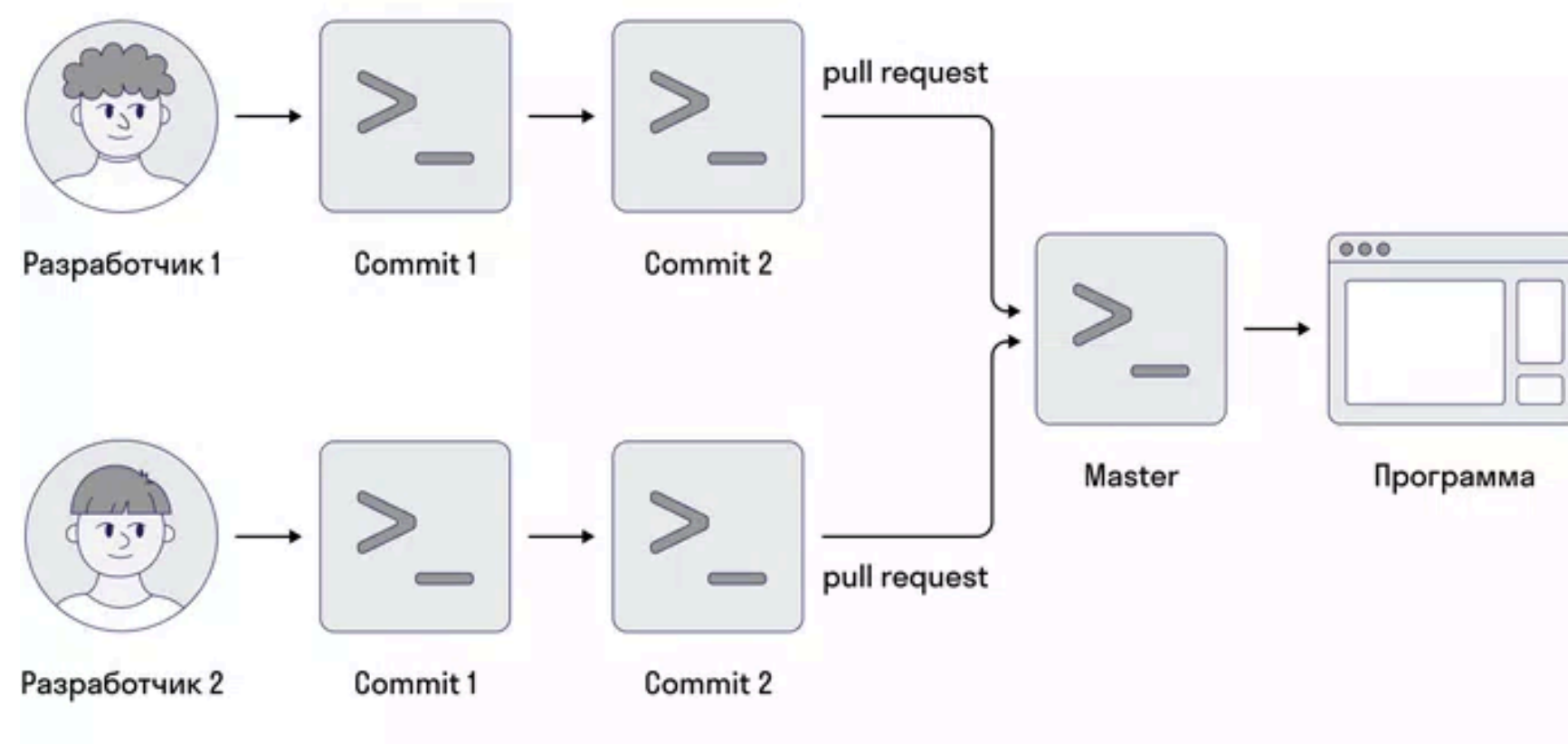
Свои изменения разработчики заливают в главную ветку — **Master**.

Мастер-ветка — ветка, в которой всегда содержится финальный, готовый, стабильный, обязательно работающий код без ошибок.

Перед тем как добавить изменения в мастер-ветку, выполняется **pull request**.

Pull request (запрос на слияние, пул-реквест или PR) — запрос на слияние веток в главной ветке.

При создании пул-реквеста владельцу репозитория приходит запрос на слияние веток: изменения рассматриваются, могут добавляться коммиты, пул-реквест принимается.



Вместо «залить изменения» еще говорят «смёржить» (от англ. to merge — сливать, объединять).

Merge — непосредственно сам процесс слияния (объединения) двух веток. При этом ветка-получатель, например мастер-ветка, получает отсутствующие коммиты от ветки-отправителя.

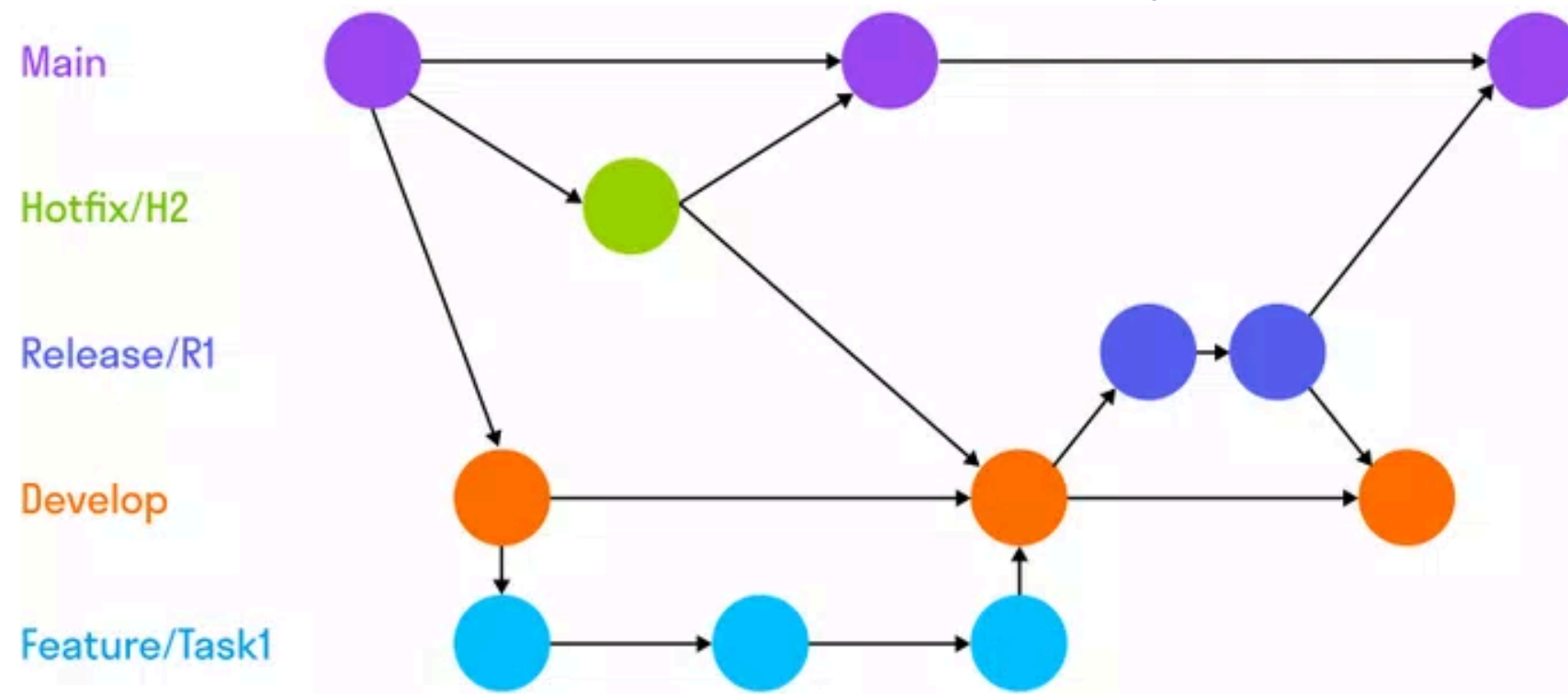
GitFlow

Какое количество веток создавать и как в них работать определяется принятой в команде методикой, одна из них — GitFlow.

GitFlow — методика работы с Git, в которой используются функциональные ветки и несколько основных, а также определяется, какие виды веток необходимы проекту и как выполнять слияние между ними.

Какие ветки обычно бывают на проекте:

- **Main (Master)** — основная ветка, которая выкатывается на продакшн.
- **Develop** — рабочая ветка, куда попадают все принятые изменения.
- **Feature/Task1** — функциональная ветка для внедрения фичи, берется из ветки develop.
- **Release/R1** — ветка для подготовки релиза.
- **Hotfix/H2** — ветка, которая используется для внесения срочных исправлений, берется от ветки main.



В ветки main и develop нельзя напрямую делать коммиты без pull request.

Основные операции с ветками

Ветки представляют собой указатель на снимок изменений. Вместо того чтобы копировать файлы из каталога в каталог, Git хранит ветку в виде ссылки на коммит. Получается, что ветка представляет собой вершину серии коммитов, а не контейнер для коммитов.

Основные команды при работе с ветками:

- **git branch** — отображение списка веток в репозитории. Это синоним команды `git branch --list`;
- **git branch <branch_name>** — создание новой ветки с именем **<branch_name>**. Эта команда не выполняет переключение на эту новую ветку;
- **git branch -d <branch_name>** — удаление указанной ветки. Это безопасная операция, поскольку Git не позволит удалить ветку, если в ней есть неслитые изменения;
- **git branch -D <branch>** — принудительное удаление указанной ветки, даже если в ней есть неслитые изменения. Эта команда используется, если вы хотите навсегда удалить все коммиты, связанные с определенным направлением разработки;
- **git branch -m <branch_new_name>** — изменение имени текущей ветки на **<branch_new_name>**;
- **git branch -a** — вывод списка всех удаленных веток;
- **git checkout <branch>** — переключение на ветку **<branch>**;
- **git checkout -b new_branch** — создание и переключение на новую ветку;
- **git pull origin <branch>** — пул (получение) ветки **<branch>** из удаленного репозиторий.
- **git push origin <branch>** — пуш (отправка) ветки **<branch>** в удаленный репозиторий.



Команда `git branch`, операции над ветками

Команда `git branch` позволяет создавать, просматривать, переименовывать и удалять ветки. Она не дает возможности переключаться между ветками или выполнять слияние разветвленной истории.

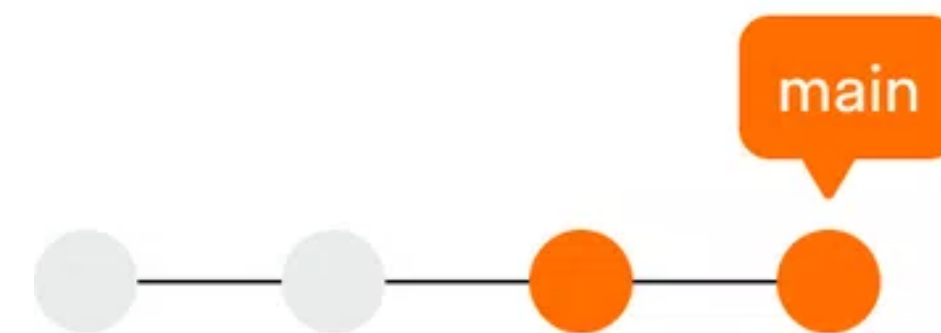
Именно поэтому команда `git branch` тесно связана с командами `git checkout` и `git merge`.

Рассмотрим подробнее работу команды `git branch`:

Создание локальных веток.

Ветки — это указатели на коммиты. Когда вы создаете ветку, Git просто создает новый указатель. Репозиторий при этом никак не изменяется.

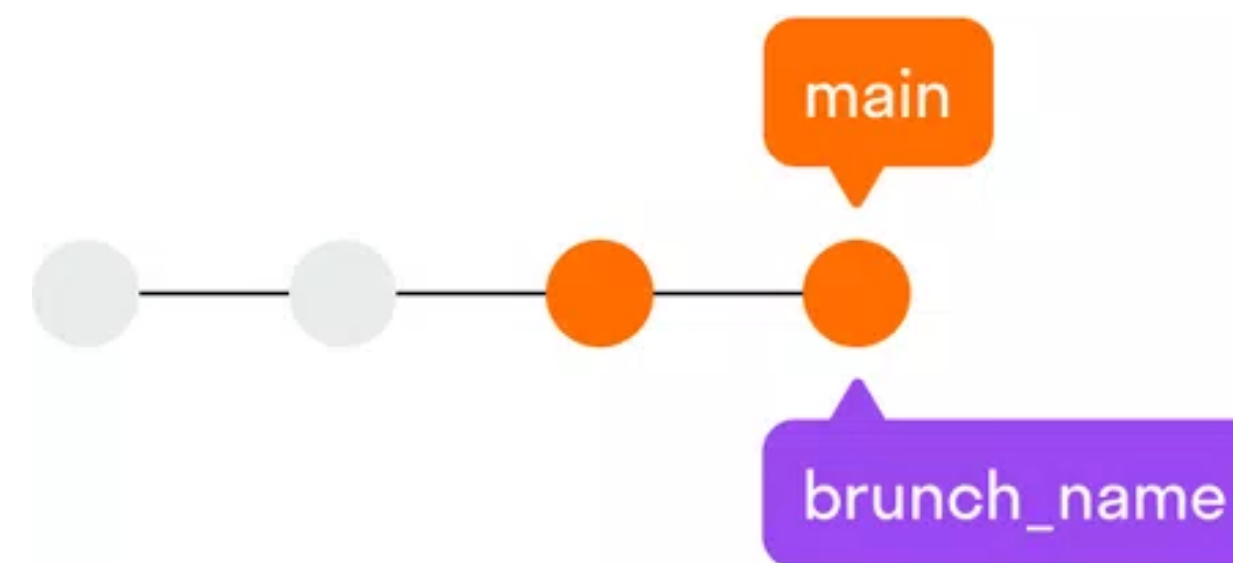
Например, вы начинаете работать с репозиторием, который выглядит так:



Затем вы создаете новую ветку:

`git branch branch_name`

История репозитория остается неизменной. Всё, что вы получаете, — это новый указатель на текущий коммит:



Команда `git branch <branch_name>` только создает новую ветку.

Чтобы добавить в эту ветку коммиты, выберите ее с помощью команды `git checkout`, а затем используйте стандартные команды `git add` и `git commit`.



Команда `git checkout`

Команда `git checkout` позволяет перемещаться между ветками, созданными командой `git branch`

При переключении ветки происходит обновление файлов в рабочем каталоге в соответствии с версией, которая хранится в этой ветке, а Git начинает записывать все новые коммиты в этой ветке.

Переключаемся на удаленную ветку

Когда вы только начинаете проект с Git, вы работаете с двумя репозиториями и ветками: локальный репозиторий с основной веткой на вашем компьютере и удаленный репозиторий с основной веткой на GitHub.

Вы можете передавать изменения из локальной основной ветки в удаленную главную ветку, а также извлекать изменения из удаленной ветки.

Когда вы создаете ветку локально, она существует только локально, пока не будет отправлена на GitHub, где она станет удаленной веткой.

Это показано в следующем примере:

Создаем и переключаемся на новую ветку

- **`git checkout -b new_branch`**

Делаем изменение

- **`touch new_file.py`**

Делаем коммит

- **`git commit -am "add new file"`**

Пуш в `new_branch` удаленного репозитория

- **`git push origin new_branch`**

В приведенном примере **`origin new_branch`** становится удаленной веткой. В коде мы создали новую ветку и зафиксировали в ней изменение перед отправкой в новую удаленную ветку и отправили ее в удаленный репозиторий

Команда `git merge`

Команда `git merge` используется для объединения двух веток.

В Git с помощью слияния собирается воедино разветвленная история.

Представим, что у нас есть новая функциональная ветка, которая отходит от главной ветки **main**. И мы хотим объединить эту функциональную ветку с **main**. При вызове команды **git merge** произойдет слияние указанной функциональной ветки с текущей — в данном случае с веткой **main**.

Коммиты слияния отличаются от других наличием двух родительских элементов. Создавая коммит слияния, Git попытается автоматически объединить две истории. Однако если Git обнаружит, что вы изменили одну и ту же часть данных в обеих историях, сделать это автоматически не удастся.

Это называется конфликтом управления версиями, и для его разрешения Git потребуются действия пользователя.

Подготовка к слиянию

Выполните команду **git status** или **git branch**. Это позволит убедиться, что **HEAD** указывает на ветку, принимающую результаты слияния.

При необходимости выполните команду **git checkout <принимающая_ветка>**, чтобы переключиться на принимающую ветку. Для примера выполним команду **git checkout main**.

Убедитесь, что в принимающей ветке и ветке для слияния содержатся последние изменения из удаленного репозитория.

Выполните команду **git fetch**, чтобы получить из него последние коммиты. Затем убедитесь, что в ветке **main** также содержатся последние изменения. Если нет, выполните команду **git pull**.

Команда `git merge`

`git pull` — это сокращение для последовательности двух команд: **`git fetch`** (получение изменений с сервера) и **`git merge`** (сливание в локальную копию).

Раздельное использование позволяет сначала просмотреть список коммитов, а не автоматически мерджить их.

Слияние делается через команду **`git merge new_branch`**, где **`new_branch`** — название ветки, которая будет объединена с принимающей, т. е. той, в которой вы сейчас.

Ускоренное слияние

Ускоренное слияние происходит, когда последний коммит текущей ветки является прямым продолжением целевой ветки.

Здесь для объединения истории Git не выполняет полноценное слияние, а переносит указатель текущей ветки в конец целевой ветки. Объединение историй проходит успешно, поскольку все коммиты целевой ветки теперь доступны из текущей ветки.

Так будет выглядеть ускоренное слияние одной из функциональных веток с веткой

Создаем feature из develop и переключаемся на новую ветку

- **`git checkout -b feature develop`**

Редактируем файлы и добавляем их в коммит

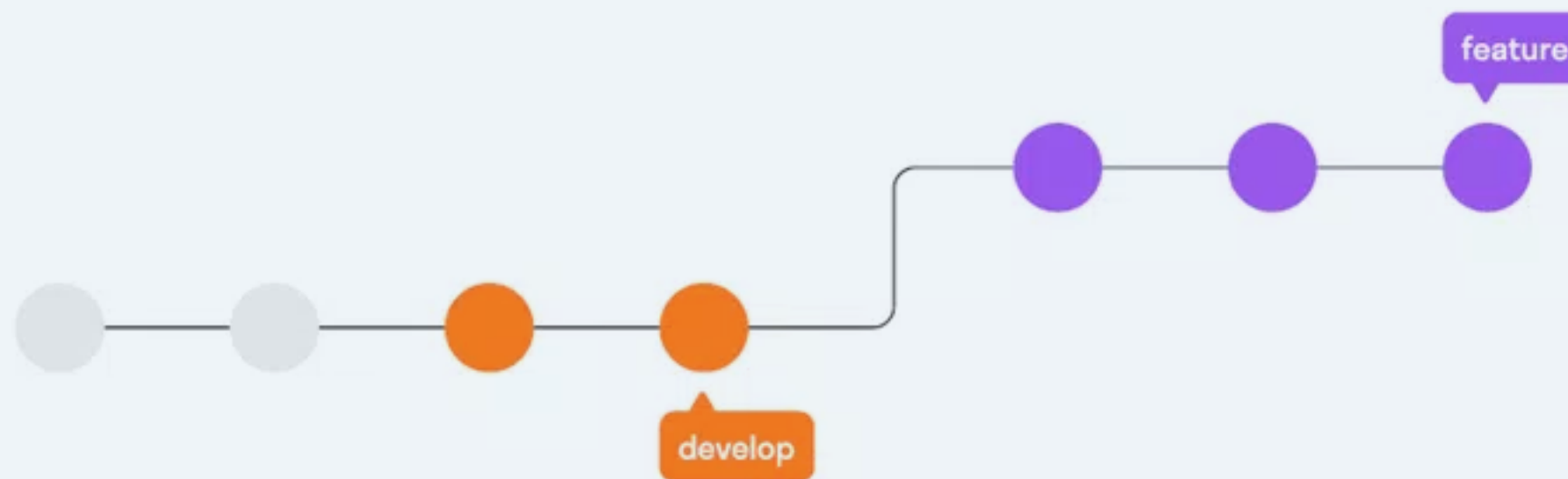
- **`git add file1 git commit -m "добавлен file1"`**

Редактируем новые файлы и добавляем их в коммит

- **`git add file2 git commit -m "добавлен file2"`**

Сливаем коммиты ветки feature в develop

- **`git checkout develop`**
- **`git merge feature git branch -d feature`**



Команда git merge

Трехстороннее слияние

Выполнить ускоренное слияние не получится, если ветки после разделения развивались независимо друг от друга. Если до целевой ветки нет прямого пути, Git будет вынужден объединить их методом трехстороннего слияния.

Метод называется трехсторонним, поскольку Git использует три коммита для создания коммита слияния — последние коммиты двух веток и общий родительский элемент.

Следующий пример похож на предыдущий, но в нем слияние должно быть трехсторонним, поскольку работа над веткой develop ведется одновременно с работой над функцией.

Так часто происходит в случае объемных функций или когда над одним проектом одновременно работают несколько разработчиков.

Создаем feature из develop и переключаемся на новую ветку

- **git checkout -b feature develop**

Редактируем файлы и добавляем их в коммит

- **git add file1**
- **git commit -m "добавлен file1"**

Редактируем новые файлы и добавляем их в коммит

- **git add file2**
- **git commit -m "добавлен file2"**

Ветка develop тоже меняется

- **git checkout develop**

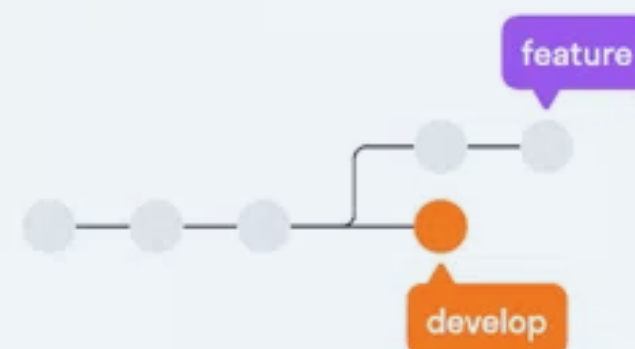
Редактируем файлы и добавляем их в коммит

- **git add file3**
- **git commit -m "Сделаны стабильные изменения в develop"**

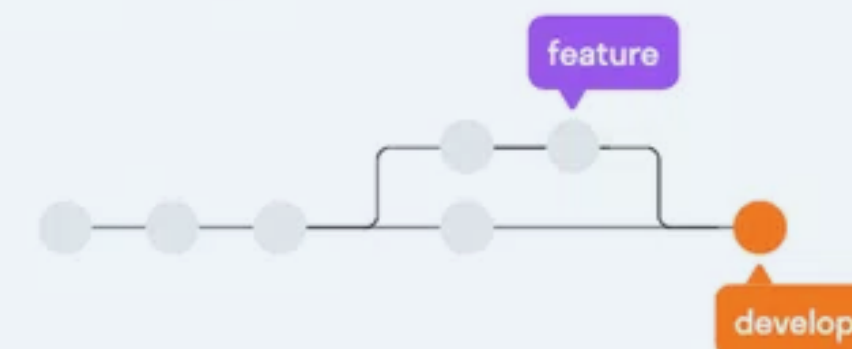
Сливаем коммиты ветки feature в develop

- **git merge feature**
- **git branch -d feature**

До git merge



После git merge





Команда git rebase

Представим ситуацию: вы работаете над функциональной веткой **feature**, при этом код в главной ветке **main** уже изменился с начала вашей работы. Вам нужно отразить последние изменения ветки **main** в ветке **feature**, не засоряя при этом историю вашей ветки, чтобы создать впечатление, словно ваша работа велась на основе последней версии **main**.

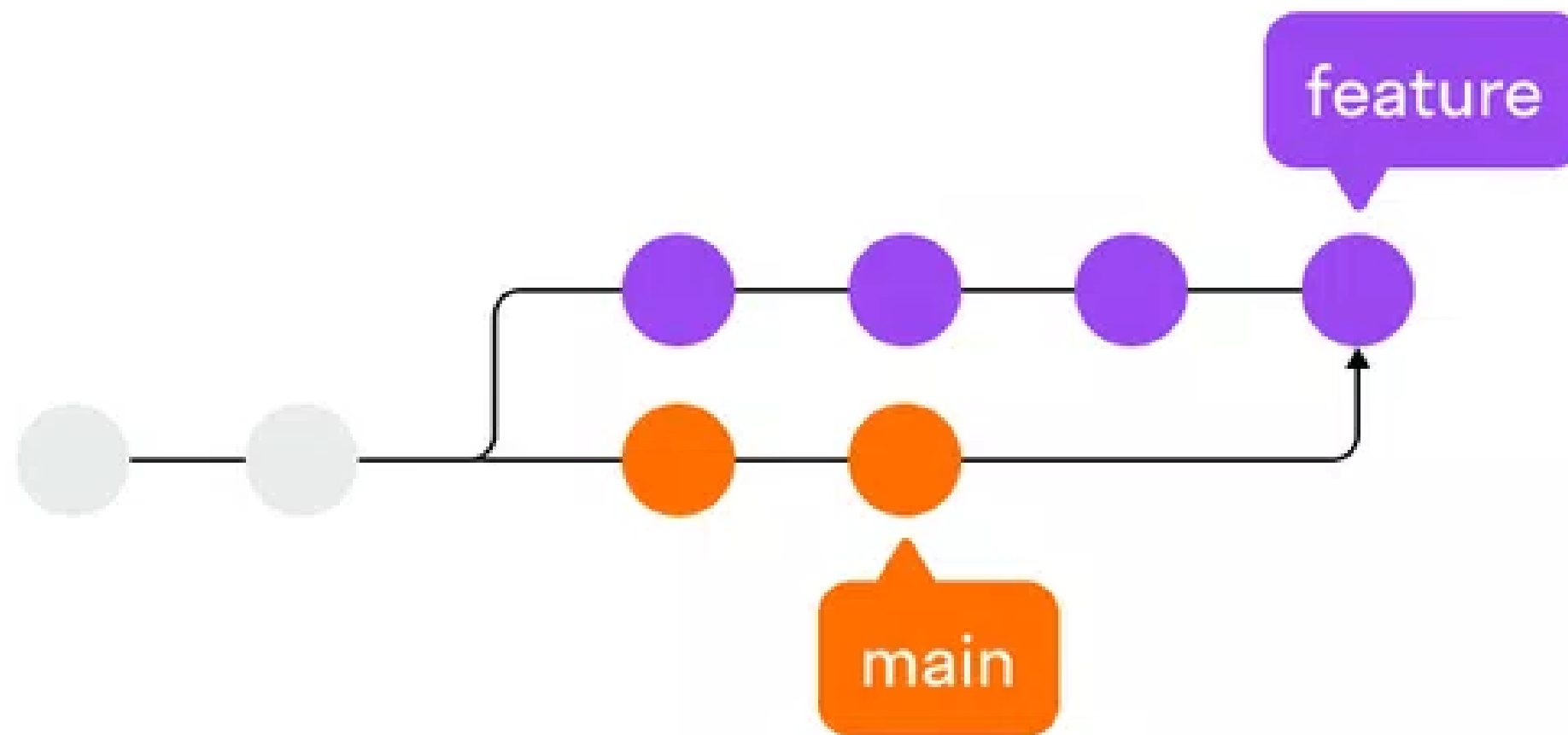
Сделать это можно так:

- **git checkout feature**
- **git merge main**

Или так:

- **git merge main feature**

При этом появится отдельный merge-коммит (коммит слияния):



Команда git rebase

При использовании для обновления ветки **feature** команды **git rebase** происходит создание новых коммитов в ветке **feature** и заполнение их свежими коммитами из **main**, а также перемещение последовательности коммитов самой ветки **feature** к новому основанию. Процесс проще понять, если сравнить две схемы:

Создаем новую ветку feature из main и переключаемся на нее

- **git checkout -b feature main**

Делаем три коммита в ветке feature

- **git add file1**
- **git commit -m "добавлен file1"**

Редактируем новые файлы и добавляем их в коммит

- **git add file2**
- **git commit -m "добавлен file2"**

Редактируем новые файлы и добавляем их в коммит

- **git add file3**
- **git commit -m "добавлен file3"**

В ветке main за время работы над feature появилось 2 новых коммита

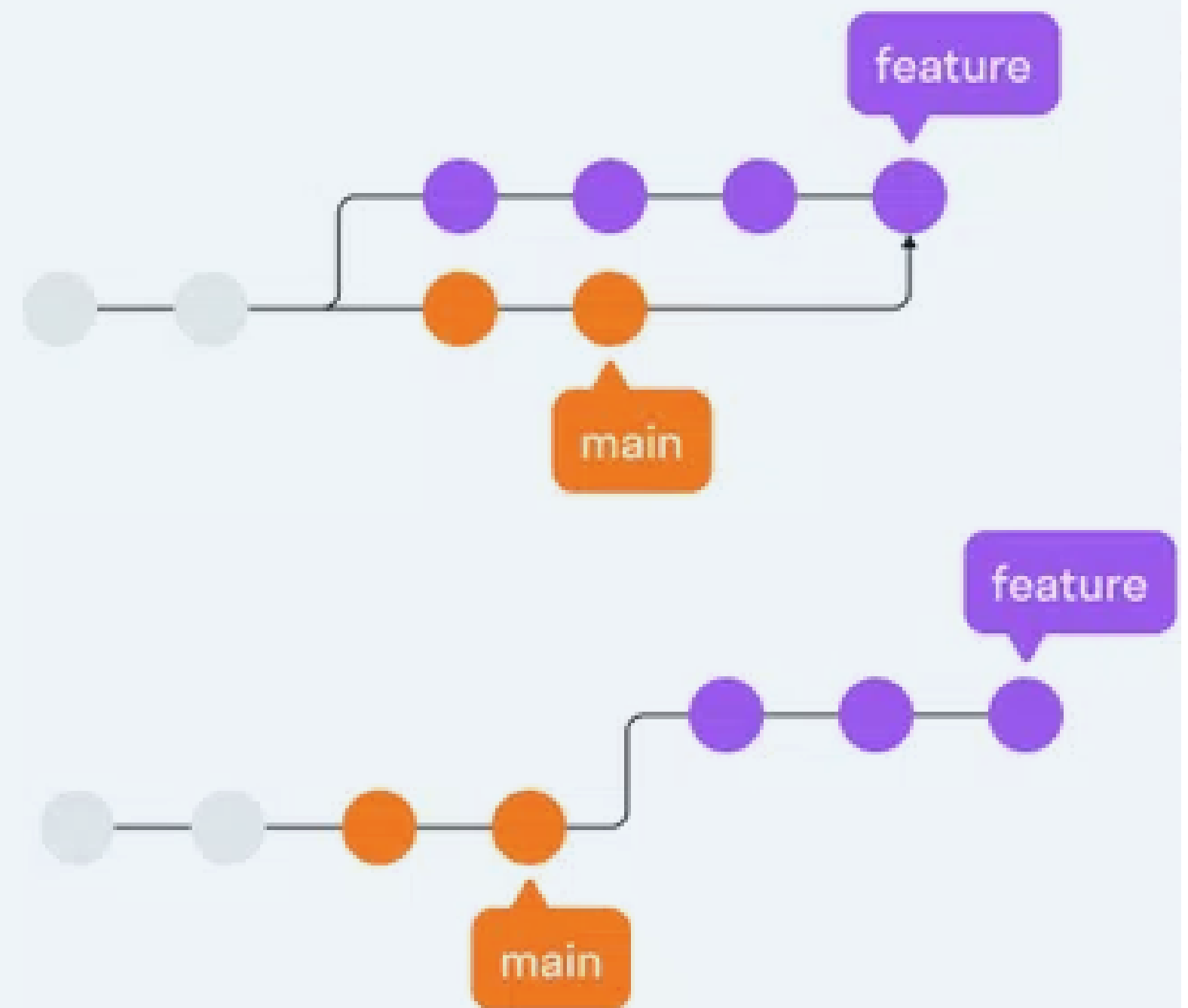
- **git checkout main**

Редактируем файлы и добавляем их в коммит

- **git add file3**
- **git commit -m "Добавлен file3 в main"**
- **git add file4**
- **git commit -m "Добавлен file4 в main"**

Обновляем ветку feature коммитами ветки

- **main git checkout feature**
- **git rebase main**



Разрешение конфликтов

При попытке объединить ветки, в которых изменена одна и та же часть того же файла, Git не сможет сделать выбор между версиями. В таком случае операция останавливается прямо перед созданием коммита слияния, чтобы пользователь вручную разрешил конфликты.

После обнаружения конфликтующих участков кода можно исправить их по своему усмотрению.

Когда вы будете готовы завершить слияние, выполните команду **git add** для конфликтующего файла или файлов — так вы сообщите Git, что конфликт разрешен.

Затем выполните обычную команду **git commit**, чтобы создать коммит слияния.

Конфликты возможны только в процессе трехстороннего слияния и не могут возникать при ускоренном слиянии.

Игнорирование файлов



.gitignore

Игнорируемые файлы — это, как правило, артефакты сборки и файлы, генерируемые машиной из исходных файлов в вашем репозитории, либо файлы, которые по какой-либо иной причине не должны попадать в коммиты.

Файл .gitignore может содержать шаблоны, которые сопоставляются с именами файлов в репозитории, чтобы определить, необходимо ли игнорировать эти файлы.

Примеры шаблонов:

- `venv` — игнорируются каталоги и файлы с именем `venv`.
- `venv/` — игнорируется содержимое любого каталога с именем `venv`.
- `/debug.log` — игнорируется `debug.log` в корне, но не в любом другом каталоге.
- `*.рус` — игнорируются все файлы с расширением `.рус`.
- `*.log` — игнорируются все файлы с расширением `.log`.
- `!debug.log` — игнорируются все логи, кроме `debug.log`.

Пример содержания файла .gitignore:

```
venv/  
.idea/  
__pycache__/  
*.рус
```